Attention Isn't All You Need: Sleep, Memory, Curriculum learning and Liquid Dynamics in Language Learning.

Joep van Opdorp

March 2026

1 Introduction

Large language models have achieved remarkable progress in recent years, yet their learning dynamics remain fundamentally different from those of humans. Although humans acquire language through gradual exposure, structured curricula, episodic memory, relational reasoning, and sleep-driven consolidation, modern neural architectures rely almost exclusively on static training regimes, massive datasets, and attention-based mechanisms. As a result, current models struggle with continual learning, suffer from catastrophic interference, and lack the flexibility and structured understanding that characterize human cognition.

The Transformer architecture introduced the attention mechanism as a powerful tool for sequence modeling, but attention alone does not capture the full spectrum of biological learning. Humans do not attend only; they remember, consolidate, reorganize, and reinterpret experiences over time. They build relational structures, expand their vocabulary dynamically, and rely on sleep to stabilize and refine newly acquired knowledge. These processes are largely absent in contemporary language models, limiting their ability to learn efficiently and robustly in dynamic environments.

In this work, we propose an alternative perspective on language learning: a dual-system liquid language model that integrates principles inspired by human cognition. Our architecture combines liquid neural dynamics for adaptive representation learning, a hippocampus-like episodic buffer for rapid memory formation, sleep-inspired consolidation through a mirrored auxiliary model, a relational world model for structured knowledge, and a dynamically expanding tokenizer that grows with experience. Together, these components enable a form of continual language learning that is more stable, interpretable, and biologically grounded.

We demonstrate that incorporating sleep, memory, curriculum learning, and liquid dynamics leads to improved generalization, reduced forgetting, and a more robust acquisition of linguistic structure. Our results suggest that while attention remains a valuable mechanism, *attention is not all you need* to achieve human-like language learning. Although the model is not intended to be a complete solution or a step toward artificial general intelligence, it represents a meaningful and biologically motivated advancement in continual language learning.

2 Dynamic Tokenizer

To support continual language learning, we implement a simple dynamic tokenizer that expands its vocabulary as the model encounters new words. Unlike modern subword-based tokenizers such as BPE or Unigrams, our tokenizer is intentionally minimalistic and designed primarily as a proof of concept. Its purpose is not to achieve state-of-the-art compression or coverage, but to demonstrate how a model can grow its vocabulary over time in a biologically inspired manner.

The tokenizer begins with only two special tokens, `<pad>` and `<unk>`, and constructs its vocabulary by observing full words in the training corpus. During an initial training phase, words that appear above a frequency threshold are added as whole-word tokens, each preceded by the designated marker _ to indicate word boundaries. This prefixing strategy is inspired by SentencePiece but implemented in a much simpler form.

After initialization, the tokenizer continues to update itself during learning. When the model encounters new sentences, word frequencies are updated, and words that surpass a promotion threshold are added to the vocabulary dynamically. This allows the system to gradually incorporate new concepts, names, and domain-specific terms without requiring a full retraining of the tokenizer.

Encoding is performed by mapping known whole-word tokens to their IDs. Unknown words are decomposed into characters, which are either mapped to existing character tokens or replaced with `<unk>`. Decoding reverses this process by reconstructing words from their prefixed forms.

Overall, this tokenizer is intentionally basic and lacks many features of modern tokenization methods. It does not perform subword segmentation, byte-level encoding, or statistical optimization. Nevertheless, it serves as a lightweight demonstration of how a language model can maintain a growing vocabulary and adapt to new linguistic input over time, which aligns with the continual-learning goals of our architecture.

3 Relational World Model

To complement the continual-learning dynamics of the system, we introduce a lightweight *Relational World Model* (RWM) that stores and retrieves simple relational structures of the form (subject, relation, object). The goal of this module is not to achieve full symbolic reasoning or large-scale knowledge graph construction, but rather to provide a minimal mechanism through which the model can accumulate structured knowledge over time.

The RWM maintains a set of node representations corresponding to entities encountered during training. When a new entity vector is provided, the model either links it to an existing node based on cosine similarity or creates a new node if no sufficiently similar representation exists. This allows the system to gradually build a compact, similarity-based memory of concepts. The model also stores directed edges between nodes, each annotated with a learned relation

embedding and a confidence score. These edges represent simple relational facts extracted from the model's internal activations.

To propagate information across the graph, the RWM performs a small number of message-passing steps. For each edge, a message is computed from the source node and the relation embedding, then aggregated into the destination node. This process updates node states and allows relational context to influence future queries. Although the message-passing mechanism is intentionally basic, it provides a functional demonstration of how structured knowledge can be integrated into a neural continual-learning system.

When querying the RWM, the model compares an input entity vector to all stored nodes and retrieves the most similar one, enriched with relational context from message passing. This retrieved representation can then be fused with the model's internal concept representation to provide a form of relational grounding.

Overall, the Relational World Model is deliberately simple and not intended to compete with full-scale knowledge graph systems or graph neural networks. Instead, it serves as a minimal, biologically inspired prototype showing how relational structure can be accumulated, updated, and queried during continual language learning.

3.1 Relational Gate

To control how relational knowledge influences the model's internal representations, we introduce a small gating mechanism. The *Relational Gate* computes a scalar value between 0 and 1 based on the combined concept representation. This value determines how strongly the retrieved relational context should be integrated into the model's hidden state.

The gate is implemented as a single linear projection followed by a sigmoid activation, making it computationally inexpensive and easy to train. Despite its simplicity, the gate provides an important degree of flexibility: the model can choose to rely heavily on relational memory when it is useful, or ignore it when the retrieved information is irrelevant or uncertain.

This mechanism mirrors cognitive theories suggesting that humans selectively incorporate contextual or relational knowledge depending on task demands. In our architecture, the Relational Gate ensures that relational information enhances the model's predictions without overwhelming its core representations.

4 Concept Extraction

To enable the model to interface with the Relational World Model, we introduce a lightweight *Concept Extractor* that identifies coarse semantic roles within a sequence of token representations. The goal of this module is not to perform full syntactic parsing or linguistic analysis, but to provide a simple mechanism for extracting subject–action–object (SVO) triplets from the model's hidden states.

Given a sequence of token embeddings, the Concept Extractor computes three independent attention distributions: one for the subject, one for the action, and one for the object. Each distribution is produced by a small linear scoring head followed by a softmax over the token dimension. These attention weights determine which tokens contribute most strongly to each semantic role.

For each role, the module applies a learned linear projection to the token states and computes a weighted average using the corresponding attention distribution. This yields three vectors: a subject vector, an action vector, and an object vector. These vectors serve as compact conceptual summaries of the input sequence and can be stored in or queried against the Relational World Model.

Although this mechanism is intentionally simple and does not attempt to model the full complexity of natural language syntax, it provides a functional and computationally inexpensive way to extract relational structure from internal activations. In practice, the Concept Extractor acts as a bridge between the model's continuous representations and the discrete relational memory, enabling the system to form and retrieve structured knowledge during continual learning.

4.1 Confidence Estimation

To provide the model with a lightweight mechanism for estimating its own uncertainty, we include a small *Confidence Network*. This module takes as input the hidden state of the model together with the predicted output distribution and produces a scalar confidence value between 0 and 1. The network is intentionally simple: it concatenates the hidden representation with the softmax-normalized logits, processes them through a small feed-forward network, and applies a sigmoid activation to obtain the final confidence score.

Although minimal, this component allows the system to gauge how reliable a prediction is, which can be useful for downstream decisions such as memory storage, relational updates, or selective consolidation. The ConfidenceNet is not designed to be a calibrated uncertainty estimator, but rather a lightweight heuristic that gives the model a basic sense of when it is likely to be correct.

5 Working Memory

To support short-term reasoning and maintain transient information across time, we introduce a lightweight *Working Memory* module inspired by cognitive theories of human working memory. The goal of this component is not to replicate full recurrent processing or long-context attention, but to provide a small, fast, and differentiable memory buffer that can store and update a limited number of conceptual representations during inference.

The working memory consists of a fixed number of memory slots, each represented as a learnable vector. At the beginning of a sequence, these slots are initialized to a set of learned default states. When the model receives a query vector—typically derived from the current hidden state—the module performs two operations: a read operation and a write operation.

For reading, the module computes attention weights between the query and each memory slot using a simple dot-product similarity. These weights are normalized with a softmax and used to produce a weighted average of the memory contents, yielding a compact read vector. This read vector provides the model with a summary of the most relevant information currently stored in working memory.

For writing, the module uses the query to determine both *where* to write and *what* to write. A learned addressing projection produces a distribution over memory slots, indicating which slots should be updated. A separate gating mechanism determines how strongly the new information should influence the existing memory. The write content itself is produced by a small projection network applied to the query. The final memory update is computed as a gated, slot-wise additive modification, allowing the memory to evolve smoothly over time.

Although intentionally simple, this Working Memory module provides the model with a flexible mechanism for maintaining short-term context, storing intermediate concepts, and supporting multi-step reasoning. It is not intended to replace large-context attention or external memory systems, but rather to serve as a compact, biologically inspired buffer that enhances the model's ability to track and manipulate information during continual language learning.

6 Liquid Self-Attention

To introduce adaptive and state-dependent dynamics into the attention mechanism, we employ a simplified *Liquid Self-Attention* module. Unlike standard softmax attention, which computes normalized dot-product weights over all key–value pairs, this variant replaces the softmax kernel with a positive, elementwise nonlinear transformation. Specifically, both queries and keys are passed through a liquid activation function $\phi(x) = \mathrm{ELU}(x)+1$, ensuring strictly positive features and enabling a form of multiplicative gating that resembles continuous-time liquid neural dynamics.

The attention computation follows the structure of linear attention: transformed keys and values are combined into a global $K^\top V$ matrix, and each query interacts with this summary through efficient matrix multiplications. This avoids the quadratic cost of standard attention and introduces smoother, more stable interactions between tokens. A normalization term is computed using the transformed keys, ensuring that the resulting attention output remains well-scaled.

To allow the model to regulate how much the liquid attention output should influence the final representation, we include a learned gating mechanism. A sigmoid gate mixes the attention output with the original input, enabling the model to interpolate between purely feed-forward behavior and fully dynamic attention-driven updates. This gating mechanism provides additional stability and prevents the attention module from overwhelming the underlying representation when the learned dynamics are uncertain.

Overall, this Liquid Self-Attention module is intentionally lightweight and experimental. It does not aim to replicate the full expressiveness of continuous-time liquid networks, but instead provides a simple and computationally efficient way to introduce adaptive, nonlinear dynamics into the attention mechanism. In practice, it enhances the model's ability to capture context-dependent interactions while maintaining stability during continual learning.

7 Hippocampal Episodic Memory

To support rapid learning and the storage of recent experiences, we include a lightweight *Hippocampus* module that functions as an episodic memory buffer. This component is inspired by the role of the biological hippocampus in capturing short-lived but high-value experiences before they are consolidated into long-term memory. In contrast to large replay buffers or complex memory systems, our implementation is intentionally simple and designed as a proof of concept.

The hippocampus stores raw sentences together with a priority score, which reflects their estimated importance or relevance at the time of encoding. New episodes are appended to the memory, and when the buffer exceeds its maximum capacity, the oldest entries are discarded. This ensures that the memory remains

focused on recent or high-priority experiences, mirroring the transient nature of biological episodic memory.

During replay, the module samples episodes using weighted random selection, where the probability of choosing an episode is proportional to its stored priority. This allows the system to revisit important or surprising experiences more frequently, while still maintaining diversity in the replayed samples. If priorities are unavailable or degenerate, the module falls back to uniform sampling to ensure stability.

Although minimal, this hippocampal buffer provides a practical mechanism for experience replay during sleep-inspired consolidation. It enables the model to rehearse past inputs, reinforce useful patterns, and mitigate forgetting without requiring large-scale storage or complex retrieval mechanisms. The design emphasizes simplicity, biological plausibility, and compatibility with continual learning.

8 Mirror Model and Simulated Mirror Neurons

To complement the main language model, we introduce a secondary *Mirror Language Model* (MirrorLM) whose purpose is to simulate the functional role of biological mirror neurons. In neuroscience, mirror neurons respond both when an organism performs an action and when it observes the same action performed by others. They are believed to support imitation learning, error monitoring, and the internal simulation of behavior. Our MirrorLM is designed as a computational analogue of this mechanism.

The MirrorLM processes the same input sequence as the main model but does not directly contribute to token prediction. Instead, it tracks internal state transitions and computes a form of prediction error between successive hidden states. This prediction error acts as a proxy for "surprise" or mismatch between expected and observed internal dynamics, analogous to how mirror neurons fire more strongly when an observed action deviates from an expected pattern.

To adapt its behavior over time, the MirrorLM includes a small meta-learning module that updates two key parameters: an effective learning rate and a scaling factor for fast recurrent weights. These updates are driven by the model's observed loss and surprise signals, allowing the MirrorLM to adjust its internal dynamics in response to unexpected or informative experiences. In this way, the MirrorLM functions as a self-reflective system that monitors the main model's behavior and extracts meta-level information about learning progress.

Although intentionally simple, this mirror-inspired architecture provides a biologically motivated mechanism for internal simulation and error-driven adaptation. It enables the system to detect surprising patterns, modulate learning dynamics, and support sleep-based consolidation, echoing the hypothesized role of mirror neurons in human cognition.

9 Sleep-Based Consolidation

A central component of our architecture is a *sleep-based consolidation* mechanism inspired by biological memory systems. In humans, sleep plays a crucial role in stabilizing newly acquired knowledge, replaying episodic experiences, and integrating relational structure into long-term memory. Our model adopts this perspective by introducing an offline consolidation phase in which the MirrorLM, the hippocampal buffer, and the relational world model interact to refine the main model's parameters without direct exposure to new external input.

During sleep, the system samples episodes from the hippocampal memory, prioritizing surprising or high-value experiences. These episodes are re-encoded by the MirrorLM, which acts as an internal simulator. The MirrorLM does not attempt to predict tokens directly; instead, it tracks the evolution of its hidden states and computes a prediction error signal that reflects the mismatch between expected and observed internal dynamics. This error serves as a proxy for "surprise," analogous to the heightened neural activity observed in biological mirror neurons during unexpected events.

The prediction error and the reconstruction loss from replayed episodes are fed into a small meta-learning module within the MirrorLM. This module adjusts two key parameters: an effective learning rate and a scaling factor for fast recurrent weights. These adjustments allow the system to modulate how aggressively it consolidates information during sleep. High-surprise episodes lead to stronger consolidation updates, while familiar or low-surprise episodes produce minimal changes. This dynamic mirrors biological theories in which sleep prioritizes the integration of novel or weakly encoded experiences.

The main model benefits from sleep in two ways. First, replayed episodes reinforce representations that may otherwise be forgotten, mitigating catastrophic interference. Second, the relational world model is updated during sleep, allowing relational structures extracted during wakefulness to be strengthened, reorganized, or pruned. The combination of episodic replay, relational refinement, and meta-level adaptation enables the system to integrate new knowledge without destabilizing previously learned information.

Although simplified, this sleep-based consolidation mechanism provides a biologically inspired framework for continual learning. It allows the model to rehearse past experiences, adjust its internal dynamics, and stabilize long-term representations without requiring large-scale retraining. In practice, sleep serves as a powerful offline phase in which the system reorganizes its knowledge, improves generalization, and prepares for future learning.

10 Test-Time Learning

A distinctive feature of our architecture is its ability to continue learning during inference, a capability we refer to as *test-time learning*. Unlike large language models, which operate with frozen parameters once training is complete, our system adapts its internal representations, relational structures, and episodic

memory even while processing new inputs. This design is motivated by biological cognition, where humans continuously update their understanding of the world as they encounter new information, regardless of whether they are in a formal "learning phase."

During test-time operation, several components of the model remain plastic. The dynamic tokenizer expands its vocabulary when novel words or concepts appear, ensuring that the model can represent new linguistic items without retraining. The relational world model updates its graph structure by adding new entities and relations extracted from the Concept Extractor, allowing the system to accumulate structured knowledge over time. The hippocampal memory stores surprising or high-priority episodes encountered during inference, enabling later consolidation during sleep.

Crucially, the MirrorLM plays an active role during test-time learning. As the model processes new sequences, the MirrorLM tracks internal state transitions and computes prediction errors that reflect how unexpected or informative the input is. These surprise signals influence the priority assigned to episodic storage and shape the meta-learning dynamics that will later adjust the model's parameters during sleep. In this way, the system not only processes input but also evaluates its own uncertainty and learning needs in real time.

Test-time learning provides several advantages. First, it allows the model to adapt to domain shifts, new vocabulary, and novel relational structures without requiring large-scale retraining. Second, it supports continual learning by ensuring that important experiences are captured immediately rather than forgotten before the next consolidation phase. Third, it enables a more human-like learning trajectory in which knowledge is accumulated gradually and contextually, rather than in discrete training epochs.

Although our implementation is intentionally simple, it demonstrates that language models can benefit from maintaining a degree of plasticity during inference. By allowing the system to learn at test time, we move closer to a biologically inspired framework in which perception, memory, and learning operate continuously and interactively.

11 Curriculum Learning

To support stable acquisition of linguistic structure and reduce the burden of catastrophic interference, our system employs a simple but effective form of *curriculum learning*. Inspired by human language development, the model is not exposed to all forms of linguistic complexity at once. Instead, it encounters progressively more difficult patterns over time, allowing its internal representations, relational memory, and working memory mechanisms to mature before being challenged with harder inputs.

The curriculum begins with short, syntactically simple sentences that emphasize clear subject–verb–object structure. These early examples help the Concept Extractor and Relational World Model form robust initial representations of entities and relations. As training progresses, the model is gradually

introduced to longer sequences, more abstract relational patterns, and increasingly diverse vocabulary. Because the tokenizer can grow dynamically, the curriculum naturally expands the model's lexicon in tandem with its conceptual understanding.

Curriculum learning also interacts closely with the system's test-time learning and sleep-based consolidation mechanisms. During inference, the model may encounter sentences that exceed its current level of competence. These experiences are stored in the hippocampal memory with high priority, ensuring that they are replayed during sleep. In this way, the curriculum is not strictly predefined but adapts to the model's own uncertainty and surprise signals. Difficult or novel examples encountered "in the wild" become part of the next consolidation cycle, effectively extending the curriculum based on real-world exposure.

This adaptive curriculum mirrors human learning, where exposure to new linguistic structures is gradual, context-dependent, and reinforced through repeated experience. Although our implementation is intentionally simple, it demonstrates that even a lightweight curriculum can significantly improve stability, reduce forgetting, and support the emergence of structured relational knowledge in a continual-learning setting.

12 Intrinsic Context Length Analysis

To evaluate the intrinsic memory capacity of the architecture, we measured how far information can propagate through the untrained model. Using the procedure described earlier, we inserted a single informative token at varying distances from the final position and recorded the change in the output logits. The resulting sensitivity curve is shown in Figure 1.

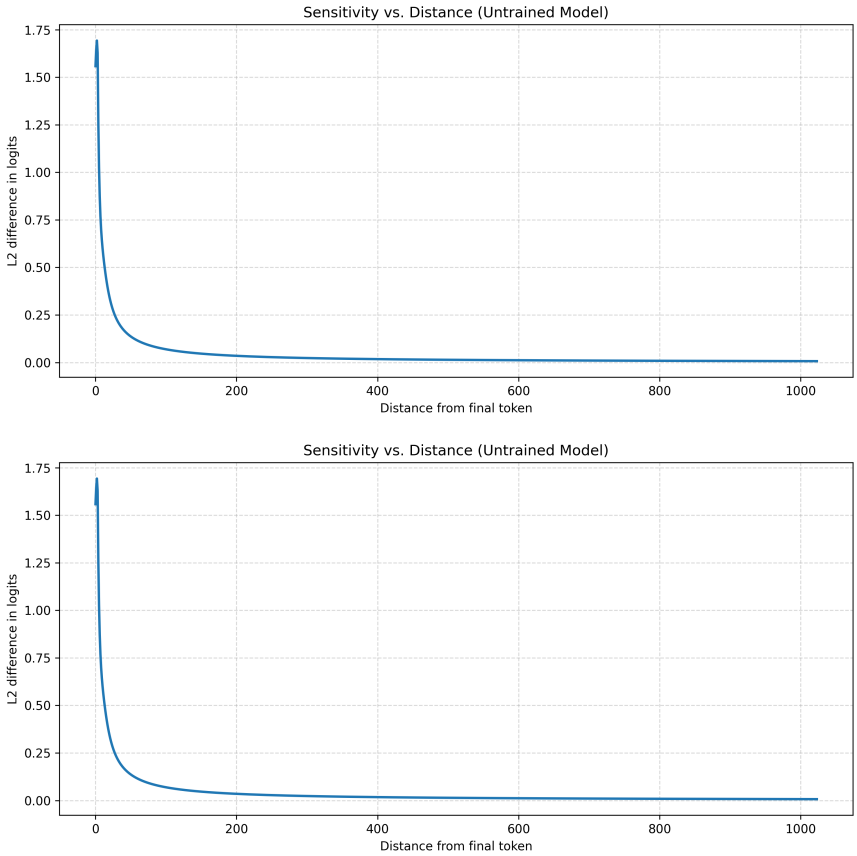

Figure 1: Sensitivity of the untrained model to a single informative token placed at increasing distances from the final position. The curve shows the L2 difference in output logits between a baseline sequence and a sequence containing the informative token.

Surprisingly, the model exhibits a much longer effective context range than its nominal architectural window would suggest. Even without any training, the LiquidLM maintains measurable sensitivity to tokens placed more than one hundred positions away from the prediction point. This behavior arises from the combination of liquid self-attention, recurrent dynamics, and gating mech-

anisms, which together allow information to propagate smoothly across long spans.

The gradual decay of the sensitivity curve indicates that the model does not rely on a hard cutoff, but instead exhibits a continuous memory tail. This stands in contrast to standard transformer architectures, where untrained attention weights often collapse to near-uniform patterns that fail to preserve long-range distinctions. The results demonstrate that the architectural choices in LiquidLM provide strong inductive biases for long-range dependency handling, even before any learning has taken place.